

DISCIPLINA: MODELAGEM E PROJETO DE BANCO DE DADOS

Professor: Romes

Atividade Prática — Parte II

Criação de Esquema de Banco de Dados no MySQL Workbench

Nome do(a) Aluno(a): GABRIEL NONATO DA SILVA

Descrição Index/View

Índice 1 (`idx_paciente_nome`): Criei esse índice na coluna nome da tabela paciente, acho que no dia a dia de uma clínica, as recepcionistas vão buscar o cadastro das pessoas principalmente pelo nome. Isso vai deixar a pesquisa muito mais rápida.

Índice 2 (`idx_medico_nome`): Da coluna nome da tabela médico, para facilitar também buscas mais rápidas pelos nomes dos profissionais.

Índice 3 (`idx_consulta_data`): Esse foi na coluna `data_hora` da tabela consulta. Uma clínica precisa o tempo todo filtrar as consultas do dia, da semana ou do mês, então esse índice vai ajudar a não ficar travando na hora de montar a agenda do dia.

View (`vw_consultas_agendadas`): Criei uma tabela virtual unindo as tabelas consulta, paciente e médico. Usei o `JOIN` entre essas 3 tabelas distintas, exatamente como foi pedido nas instruções. Essa view serve para mostrar de forma fácil o nome do paciente, o nome do médico e o horário da consulta, sem que a gente precise ficar digitando um código SQL gigante com vários joins toda vez que quiser olhar a agenda.

Escolhas de Tipos de Dados

Usei o tipo inteiro para as chaves primárias e estrangeiras, como a instrução do trabalho mandou usar `AUTO_INCREMENT`, o `INT` é um bom tipo para gerar esses números de ID sequenciais.

Usei o `VARCHAR` para coisas que tem tamanho diferente, como nome, endereço, e-mail e bairro. Assim, o banco só gasta o espaço que realmente for digitado.

Usei o `CHAR` para campos que têm um tamanho padrão, como o CPF, a UF e o CEP. Como o tamanho não muda, processa isso mais rápido.

Usei `DECIMAL` no valor da consulta e no `preco_unitario` do medicamento. Como envolve dinheiro, o formato decimal com duas casas é mais seguro para não dar problemas.

Usei `DATE` para `data_nascimento` e `data_adesao` porque só precisa saber o dia, mas para a consulta e o prontuário, usei `DATETIME` porque o horário exato que a consulta vai acontecer ou que o registro foi feito é importante.

Usei `TEXT` nos campos de sintomas, diagnostico e observacoes do prontuário porque os médicos podem escrever textos longos, e o `VARCHAR` poderia cortar a anotação.

Restrições

Coloquei PRIMARY KEY e AUTO_INCREMENT em praticamente todas as tabelas, conforme as especificações do projeto, para garantir que cada registro tenha uma identificação única.

Usei NOT NULL em campos essenciais, por exemplo, não faz sentido cadastrar um paciente sem nome e sem CPF, ou marcar uma consulta sem saber a data.

Apliquei UNIQUE no CPF, E-mail, CRM e CNPJ, para que a secretária não cadastre a mesma pessoa ou clínica duas vezes por engano.

Usei CHECK para fazer validações lógicas diretas no banco, por exemplo, no campo sexo, a regra CHECK não deixa que alguém digite qualquer outra letra. Também usei CHECK nos valores financeiros para não deixar lançarem consultas ou remédios com preços negativos.

Usando DEFAULT para a consulta, defini o status como agendada por padrão, assim a recepcionista nem precisa digitar isso quando marcar um horário, agora para o prontuário, a data_registro já pega a data e hora do servidor usando o CURRENT_TIMESTAMP.

E por último usei FOREIGN KEY para o padrão fk_tabela_origem e tabela_destino, um detalhe que eu pesquisei se existia, é que na tabela de endereço eu usei ON DELETE CASCADE, que se um paciente for excluído do sistema, o endereço dele some junto, não deixando dados perdidos no banco e no final existia mesmo.